Boolean Satisfiability

Edward A. Hirsch

http://edwardahirsch.github.io/edwardahirsch

Ariel University

January 22, 2024

Lecture Plan

- Exponential-Time Hypothesis
  - Sparsification: Equivalence w.r.t. n and m
  - SERF-completeness
- Strong Exponential-Time Hypothesis
  - Linear-size **SAT**
  - Unique $\boldsymbol{k}$-**SAT**
  - $\boldsymbol{k}$-**SAT** when $k \to \infty$

Russell Impagliazzo, Ramamohan Paturi, Francis Zane:
Which Problems Have Strongly Exponential Complexity?

Russell Impagliazzo, Ramamohan Paturi:
*On the Complexity of **k-SAT***

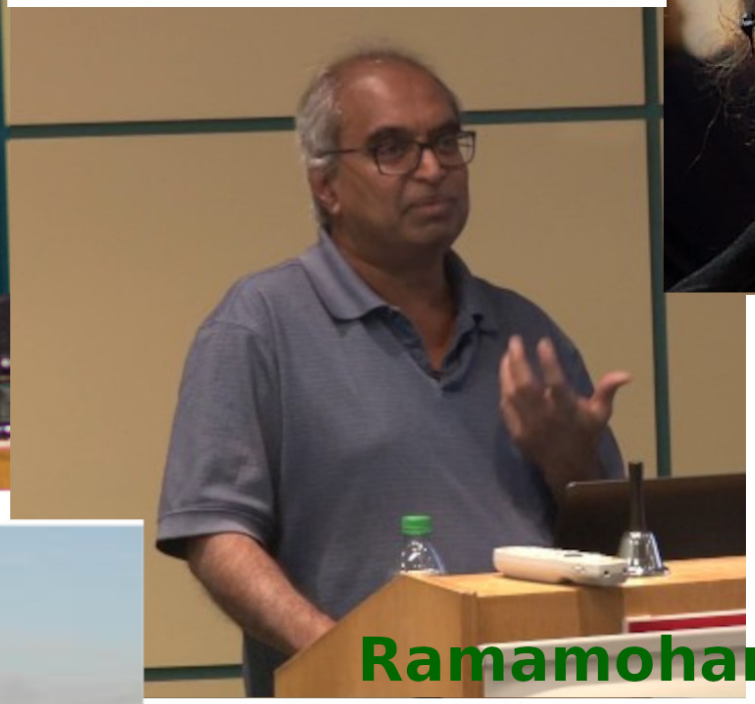
Chris Calabro, Russell Impagliazzo, Valentine Kabanets, Ramamohan Paturi:
*The Complexity of Unique-**k-SAT**: An Isolation Lemma for k-CNFs*

Chris Calabro, Russell Impagliazzo, Valentine Kabanets, Ramamohan Paturi:
*A duality between clause width and clause density for **SAT***

The Authors

Russell Impagliazzo

Valentine Kabanets



Ramamohan Paturi

Chris Calabro

Francis Zane

The Exponential-Time Hypothesis

- **3-SAT** in time 2^n, 1.5^n, $1.34^n, \ldots, 2^{n/10}, 2^{n/100}$?..
- The Exponential-Time Hypothesis (informal): NO.

The Exponential-Time Hypothesis

- **3-SAT** in time 2^n, 1.5^n, $1.34^n, \ldots, 2^{n/10}, 2^{n/100}$?..
- The Exponential-Time Hypothesis (informal): NO.

Definition

Parameterized **NP** problem (L, q):
- a language $L \in \mathbf{NP}$ (defined by relation R s.t. $x \in L \iff \exists y R(x, y)$)
- a complexity parameter $q(\ldots)$ that bounds the size of the shortest solution:

$$x \in L \iff \exists y(|y| \leqslant q(x) \wedge R(x, y)).$$

For example:
- $(\boldsymbol{k}\text{-}\mathbf{SAT}, n)$, where n is the number of variables,
- $(\boldsymbol{k}\text{-}\mathbf{SAT}, m)$, where m is the number of clauses.

The Exponential-Time Hypothesis

- **3-SAT** in time 2^n, 1.5^n, $1.34^n, \ldots, 2^{n/10}, 2^{n/100}$?..
- The Exponential-Time Hypothesis (informal): NO.

Definition

Parameterized **NP** problem (L, q):
- a language $L \in \mathbf{NP}$ (defined by relation R s.t. $x \in L \iff \exists y R(x, y)$)
- a complexity parameter $q(\ldots)$ that bounds the size of the shortest solution:

$$x \in L \iff \exists y(|y| \leqslant q(x) \wedge R(x, y)).$$

For example:
- $(\boldsymbol{k}\text{-}\mathbf{SAT}, n)$, where n is the number of variables,
- $(\boldsymbol{k}\text{-}\mathbf{SAT}, m)$, where m is the number of clauses.

Definition

A parameterized problem $(L, \boldsymbol{q}) \in \mathbf{SUBEXP}$ if
for every positive integer $\boldsymbol{t}$, the problem $x \in L$ is decidable in time $\widetilde{O}(\mathbf{2}^{\boldsymbol{q}(\boldsymbol{x})/\boldsymbol{t}})$.

An exponential time hypothesis for a problem (L, q) says that $(L, q) \notin \mathbf{SUBEXP}$.
The Exponential-Time Hypothesis, ETH: $(\mathbf{3\text{-}SAT}, n) \notin \mathbf{SUBEXP}$.

So many exponents!

If we believe in ETH...

Randomized, one-sided error 1/2

$$s_k \quad = \quad \inf\{\delta \geqslant 0 \mid \boldsymbol{k}\text{-}\mathbf{SAT} \in \ \mathbf{RTIME}[2^{\delta n}]\},$$

Reformulated: ETH : $s_3 > 0$.

So many exponents!

If we believe in ETH...

Randomized, one-sided error 1/2

$$s_k = \inf\{\delta \geqslant 0 \mid \boldsymbol{k}\text{-}\mathbf{SAT} \in \mathbf{RTIME}[2^{\delta n}]\},$$

Reformulated: ETH : $s_3 > 0$.

$$\begin{aligned}
s_k^{\text{dens.}d} &= \inf\{\delta \geqslant 0 \mid \boldsymbol{k}\text{-}\mathbf{SAT} \text{ with at most } dn \text{ clauses} \in \mathbf{RTime}[2^{\delta n}]\},\\
s^{\text{dens.}d} &= \inf\{\delta \geqslant 0 \mid \mathbf{SAT} \text{ with at most } dn \text{ clauses} \in \mathbf{RTime}[2^{\delta n}]\},\\
\sigma_k &= \inf\{\delta \geqslant 0 \mid \text{Unique } \boldsymbol{k}\text{-}\mathbf{SAT} \in \mathbf{RTime}[2^{\delta n}]\},
\end{aligned}$$

So many exponents!

If we believe in ETH...

Randomized, one-sided error 1/2

$$s_k = \inf\{\delta \geqslant 0 \mid \boldsymbol{k}\text{-}\mathbf{SAT} \in \mathbf{RTIME}[2^{\delta n}]\},$$

Reformulated: ETH : $s_3 > 0$.

$$\begin{aligned} s_k^{\text{dens.}d} &= \inf\{\delta \geqslant 0 \mid \boldsymbol{k}\text{-}\mathbf{SAT} \text{ with at most } dn \text{ clauses} \in \mathbf{RTime}[2^{\delta n}]\}, \\ s^{\text{dens.}d} &= \inf\{\delta \geqslant 0 \mid \mathbf{SAT} \text{ with at most } dn \text{ clauses} \in \mathbf{RTime}[2^{\delta n}]\}, \\ \sigma_k &= \inf\{\delta \geqslant 0 \mid \text{Unique } \boldsymbol{k}\text{-}\mathbf{SAT} \in \mathbf{RTime}[2^{\delta n}]\}, \end{aligned}$$

Where does it go?.. (Recall: **SAT** in $2^{1-1/O(\log(m/n))}$.)

$$s_\infty = \lim_{k\to\infty} s_k,$$

Strong ETH, SETH : $s_\infty = 1$.

So many exponents!

If we believe in ETH...

Randomized, one-sided error 1/2

$$s_k = \inf\{\delta \geqslant 0 \mid \boldsymbol{k}\textbf{-SAT} \in \textbf{RTIME}[2^{\delta n}]\},$$

Reformulated: ETH : $s_3 > 0$.

$$\begin{aligned} s_k^{\text{dens}.d} &= \inf\{\delta \geqslant 0 \mid \boldsymbol{k}\textbf{-SAT} \text{ with at most } dn \text{ clauses} \in \textbf{RTime}[2^{\delta n}]\}, \\ s^{\text{dens}.d} &= \inf\{\delta \geqslant 0 \mid \textbf{SAT} \text{ with at most } dn \text{ clauses} \in \textbf{RTime}[2^{\delta n}]\}, \\ \sigma_k &= \inf\{\delta \geqslant 0 \mid \text{Unique } \boldsymbol{k}\textbf{-SAT} \in \textbf{RTime}[2^{\delta n}]\}, \end{aligned}$$

Where does it go?.. (Recall: **SAT** in $2^{1-1/O(\log(m/n))}$.)

$$s_\infty = \lim_{k\to\infty} s_k,$$

Strong ETH, SETH : $s_\infty = 1$.

$$\begin{aligned} s^{\text{dens}.\infty} &= \lim_{d\to\infty} s^{\text{dens}.d}, \\ \sigma_\infty &= \lim_{k\to\infty} \sigma_k. \end{aligned}$$

How are all these constants related?

SERF Reducibilities: $(A, p) \longrightarrow (B, q)$

- $2^{\varepsilon \boldsymbol{m}}$ vs $2^{\varepsilon \boldsymbol{n}}$: If $(\boldsymbol{k}\text{-}\mathbf{SAT}, m) \in \mathbf{SUBEXP}$, does $(\boldsymbol{k}\text{-}\mathbf{SAT}, n) \in \mathbf{SUBEXP}$?
- $(\boldsymbol{k}\text{-}\mathbf{SAT}, m)$ could be easier: typically $m \geqslant n$, better c^m algorithms exist.

SERF Reducibilities: $(A, p) \longrightarrow (B, q)$

- $2^{\varepsilon \boldsymbol{m}}$ vs $2^{\varepsilon \boldsymbol{n}}$: If $(\boldsymbol{k}\text{-}\mathbf{SAT}, m) \in \mathbf{SUBEXP}$, does $(\boldsymbol{k}\text{-}\mathbf{SAT}, n) \in \mathbf{SUBEXP}$?
- $(\boldsymbol{k}\text{-}\mathbf{SAT}, m)$ could be easier: typically $m \geqslant n$, better c^m algorithms exist.

Definition (Subexponential reduction family, SERF)

SERF from (A, p) to (B, q) is a series $\{T_t^\bullet\}_{t \in \mathbb{N}}$ of oracle Turing machines s.t.

- $T_t^B(F)$ solves the problem $F \in A$ in time $\widetilde{O}(2^{p(F)/t})$,
- it asks the oracle about $G \overset{?}{\in} B$ with $q(G) = O(p(F))$ and $|G| = |F|^{O(1)}$.

Indeed, if $(B, q) \in \mathbf{SUBEXP}$, then $(A, p) \in \mathbf{SUBEXP}$.

`Run` $T_{t'}(F)$ `asking` M_B `about` G_i`'s with` $q(G_i) \leqslant ap(F)$

SERF Reducibilities: $(A, p) \longrightarrow (B, q)$

- $2^{\varepsilon \boldsymbol{m}}$ vs $2^{\varepsilon \boldsymbol{n}}$: If $(\boldsymbol{k}\text{-}\mathbf{SAT}, m) \in \mathbf{SUBEXP}$, does $(\boldsymbol{k}\text{-}\mathbf{SAT}, n) \in \mathbf{SUBEXP}$?
- $(\boldsymbol{k}\text{-}\mathbf{SAT}, m)$ could be easier: typically $m \geqslant n$, better c^m algorithms exist.

Definition (Subexponential reduction family, SERF)

SERF from (A, p) to (B, q) is a series $\{T_t^\bullet\}_{t \in \mathbb{N}}$ of oracle Turing machines s.t.

- $T_t^B(F)$ solves the problem $F \in A$ in time $\widetilde{O}(2^{p(F)/t})$,
- it asks the oracle about $G \overset{?}{\in} B$ with $q(G) = O(p(F))$ and $|G| = |F|^{O(1)}$.

Indeed, if $(B, q) \in \mathbf{SUBEXP}$, then $(A, p) \in \mathbf{SUBEXP}$.

Given an $\widetilde{O}(2^{q(G)/s})$-time machine M_B for (B, q),
Solve $A(F)$ in time $\widetilde{O}(2^{p(F)/t})$:

`Run` $T_{t'}(F)$ `asking` M_B `about` G_i`'s with` $q(G_i) \leqslant ap(F)$

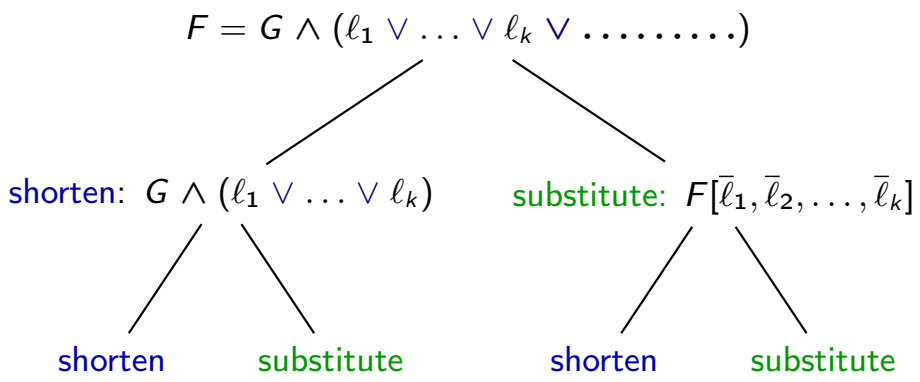
The total time (under $\widetilde{O}$):

$$2^{p(F)/t'} \cdot 2^{q(G)/s} \leqslant 2^{p(F)/t'} \cdot 2^{cp(F)/s} \leqslant 2^{p(F)/t}$$

when $t' = 2t$, $s = 2ct$.

Shortening a k-CNF: The Idea

Recall: Clause Shortening.



What if there are many clauses containing $(\ell_1 \vee \ldots \vee \ell_k)$?
We get rid of all of them simultaneously!

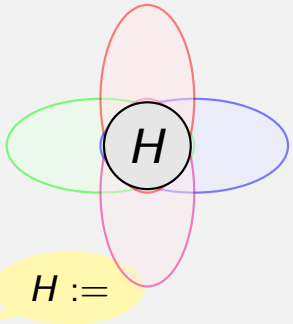
What if there are none?
Perhaps, there are few clauses left.

$(k\text{-}\mathbf{SAT}, m) \longrightarrow (k\text{-}\mathbf{SAT}, n)$, making $m = O(n)$

Algorithm (The sparsification procedure)

Input: *k-CNF formula F, small ε.*

Output: *linear-size k-CNF formulas $F_1, \ldots, F_L$ including a satisfiable formula iff F is satisfiable.*



If F contains c-clauses $C_1, \ldots, C_d$ s.t. $|\bigcap_{i=1}^{d} C_i| = h$

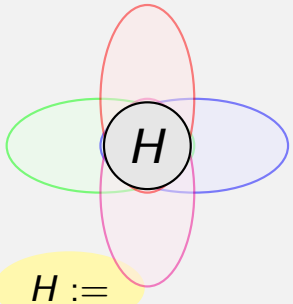
2. Make a recursive call for $F \setminus \{C_1, \ldots, C_d\} \cup \{H\}$.
3. Make a recursive call for $F[\,\overline{H}\,]$.

$(k\text{-}\mathbf{SAT}, m) \longrightarrow (k\text{-}\mathbf{SAT}, n)$, making $m = O(n)$

Algorithm (The sparsification procedure)

Input: *k-CNF formula F, small ε.*

Output: *linear-size k-CNF formulas $F_1, \ldots, F_L$ including a satisfiable formula iff F is satisfiable.*



1. For $c = 1, \ldots, n$ // the shorter, the better
   For $h = c, \ldots, 1$ // the larger, the better
   If F contains c-clauses $C_1, \ldots, C_d$ $(d \geqslant \theta_{c-h})$ s.t. $H := \left|\bigcap_{i=1}^{d} C_i\right| = h$
   Then let $H = \bigcap_{i=1}^{d} C_i$ and proceed to step 2
   If nothing is found, then return F
2. Make a recursive call for $F \setminus \{C_1, \ldots, C_d\} \cup \{H\}$.
3. Make a recursive call for $F[\,\overline{H}\,]$.

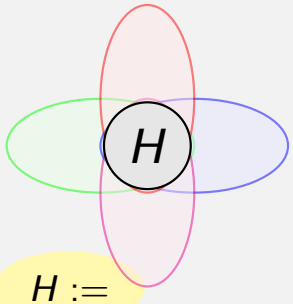
$(k\text{-}\mathbf{SAT}, m) \longrightarrow (k\text{-}\mathbf{SAT}, n)$, making $m = O(n)$

Algorithm (The sparsification procedure)

Input: *k-CNF formula F, small ε.*

Output: *linear-size k-CNF formulas $F_1, \ldots, F_L$ including a satisfiable formula iff F is satisfiable.*

Parameters: *integers $\theta_i = \ldots$*



1. For $c = 1, \ldots, n$ // the shorter, the better
   For $h = c, \ldots, 1$ // the larger, the better
   If F contains c-clauses $C_1, \ldots, C_d$ $(d \geqslant \theta_{c-h})$ s.t. $H := \left|\bigcap_{i=1}^{d} C_i\right| = h$
   Then let $H = \bigcap_{i=1}^{d} C_i$ and proceed to step 2
   If nothing is found, then return F
2. Make a recursive call for $F \setminus \{C_1, \ldots, C_d\} \cup \{H\}$.
3. Make a recursive call for $F[\,\overline{H}\,]$.

Claim: $L \leqslant 2^{\varepsilon n}$, and for every F_i, every variable occurs in it $\leqslant (k/\varepsilon)^{3k}$ times.

If so, $|F_i| \leqslant (k/\varepsilon)^{3k} n$. Take $t = (\varepsilon + (k/\varepsilon))^{3k} s$.

Given $2^{m/t}$-time k-**SAT** algorithm, solve k-**SAT** in time $2^{n/s}$:

$$2^{\varepsilon n} \cdot 2^{(k/\varepsilon)^{3k} n/t} \leqslant 2^{n \cdot (\varepsilon + (k/\varepsilon)^{3k})/t} \leqslant 2^{n/s}.$$

s_∞ vs $s^{\text{dens.}\infty}$

Sparsification produces $\leqslant 2^{\varepsilon n}$ formulas F_i,
every variable has $\leqslant (k/\varepsilon)^{3k}$ occurences in F_i.

Sparsification gives us

$$s_k \leqslant s_k^{\text{dens.}((k/\varepsilon)^{3k})} + \varepsilon.$$

Let $\varepsilon = k^{-O(1)}$, then

$$s_\infty \leqslant s^{\text{dens.}\infty}.$$

s_∞ vs $s^{\text{dens.}\infty}$

Sparsification produces $\leqslant 2^{\varepsilon n}$ formulas F_i,
every variable has $\leqslant (k/\varepsilon)^{3k}$ occurences in F_i.

Sparsification gives us

$$s_k \leqslant s_k^{\text{dens.}((k/\varepsilon)^{3k})} + \varepsilon.$$

Let $\varepsilon = k^{-O(1)}$, then

$$s_\infty \stackrel{?}{=} s^{\text{dens.}\infty}.$$

s_∞ vs $s^{\text{dens.}\infty}$

Sparsification produces $\leqslant 2^{\varepsilon n}$ formulas F_i,
every variable has $\leqslant (k/\varepsilon)^{3k}$ occurences in F_i.

Sparsification gives us

$$s_k \leqslant s_k^{\text{dens.}((k/\varepsilon)^{\mathbf{3}k})} + \varepsilon.$$

Let $\varepsilon = k^{-O(1)}$, then

$$s_\infty \leqslant s^{\text{dens.}\infty}.$$

Now prove the opposite:
Recall that the clause-shortening **SAT** algorithm runs in time

$$2^{c_k \boldsymbol{n} + 4\boldsymbol{m}/2^{c_k k}}.$$

Solve the leaves (***k*-SAT**) in $2^{(s_k+\varepsilon)n}$-time, $c_k = s_k + \varepsilon$.

$$s^{\text{dens.}d} \leqslant s_k + \varepsilon + \frac{4d}{2^{k(s_k+\varepsilon)}} \underset{\varepsilon\to 0}{\leqslant} s_k + \frac{4d}{2^{ks_k}}.$$

Take $k = k(d)$ s.t. $+\ldots \leqslant s_\infty - s_k$, take $d \to \infty$:

$$s^{\text{dens.}\infty} \leqslant s_\infty$$

s_k strictly increases i.o.

- I said $0 < \ldots \leqslant s_\infty - s_k$?

s_k strictly increases i.o.

- I said $0 < \ldots \leqslant s_\infty - s_k$?

Theorem (Impagliazzo, Paturi)

$s_k \leqslant (1 - \Omega(k^{-1}))s_\infty$.

Proof.

Sparsification and critical clauses.

□

SERF-completeness

Theorem

$(\textbf{\textit{3-SAT}}, m)$ *and* $(\textbf{\textit{3-SAT}}, n)$ *are SERF-complete for* ***SNP****.*

SERF-completeness

Theorem

*($\textbf{3-SAT}, m$) and ($\textbf{3-SAT}, n$) are SERF-complete for **SNP**.*

Definition (parameterized version of SNP – difficult!)

$(L, q) \in$ **SNP** if L can be defined as

$$(R_1, \ldots, R_u) \in L \iff \exists f_1, \ldots, f_s \; \forall i_1, \ldots, i_t \; \Phi(i_1, \ldots, i_t),$$

where

- f_j's are relations (arity k_j),
- $i_j \in [1..d]$,
- formula Φ applies f_j's and R_j's to i_j's.

The parameter q is the total bit-size of f_j's, i.e., $\sum_j d^{k_j}$.

SNP: example

Definition (parameterized version of SNP)

$(L, q) \in$ **SNP** if L can be defined as

$$(R_1, \ldots, R_u) \in L \iff \exists f_1, \ldots, f_s \; \forall i_1, \ldots, i_t \; \Phi(i_1, \ldots, i_t),$$

where f_j's are relations (arity k_j), $i_j \in [1..d]$, formula Φ applies f_j's and R_j's to i_j's.
$q((R_1, \ldots, R_u))$ is the total bit-size of f_j's, i.e., $\sum_j d^{k_j}$.

- $L =$ ***k*-SAT**, parameter n,
- $\boldsymbol{i}_1, \ldots, \boldsymbol{i}_t \in [1 \,.\,.\, n]$, counters for **variables indices**,

SNP: example

Definition (parameterized version of SNP)

$(L, q) \in$ **SNP** if L can be defined as

$$(R_1, \ldots, R_u) \in L \iff \exists f_1, \ldots, f_s \; \forall i_1, \ldots, i_t \; \Phi(i_1, \ldots, i_t),$$

where f_j's are relations (arity k_j), $i_j \in [1..d]$, formula Φ applies f_j's and R_j's to i_j's.
$q((R_1, \ldots, R_u))$ is the total bit-size of f_j's, i.e., $\sum_j d^{k_j}$.

- $L = \boldsymbol{k}$**-SAT**, parameter n,
- $\boldsymbol{i}_1, \ldots, \boldsymbol{i}_t \in [1 \,.\,.\, n]$, counters for **variables indices**,
- input relations R_i: clauses incidence, such as
  $R_{+-+--}(i_1, i_2, i_3, i_4, i_5) = \texttt{True} \iff (x_{i_1} \vee \overline{x}_{i_2} \vee x_{i_3} \vee \overline{x}_{i_4} \vee \overline{x}_{i_5}) \in F$,

SNP: example

Definition (parameterized version of SNP)

$(L, q) \in$ **SNP** if L can be defined as

$$(R_1, \ldots, R_u) \in L \iff \exists f_1, \ldots, f_s \; \forall i_1, \ldots, i_t \; \Phi(i_1, \ldots, i_t),$$

where f_j's are relations (arity k_j), $i_j \in [1..d]$, formula Φ applies f_j's and R_j's to i_j's.
$q((R_1, \ldots, R_u))$ is the total bit-size of f_j's, i.e., $\sum_j d^{k_j}$.

- $L =$ ***k*-SAT**, parameter n,
- $\boldsymbol{i}_1, \ldots, \boldsymbol{i}_t \in [1 \ldots n]$, counters for **variables indices**,
- input relations R_i: clauses incidence, such as
  $R_{+-+--}(i_1, i_2, i_3, i_4, i_5) = \texttt{True} \iff (x_{i_1} \vee \overline{x}_{i_2} \vee x_{i_3} \vee \overline{x}_{i_4} \vee \overline{x}_{i_5}) \in F$,
- solution relation $\boldsymbol{f}$ (just unary!): **a satisfying assignment**, such as
  $f(1) = \texttt{True}$, $f(2) = \texttt{False}, \ldots, f(n) = \texttt{False}$,

SNP: example

Definition (parameterized version of SNP)

$(L, q) \in$ **SNP** if L can be defined as

$$(R_1, \ldots, R_u) \in L \iff \exists f_1, \ldots, f_s \; \forall i_1, \ldots, i_t \; \Phi(i_1, \ldots, i_t),$$

where f_j's are relations (arity k_j), $i_j \in [1..d]$, formula Φ applies f_j's and R_j's to i_j's.
$q((R_1, \ldots, R_u))$ is the total bit-size of f_j's, i.e., $\sum_j d^{k_j}$.

- $L =$ ***k*-SAT**, parameter n,
- $\boldsymbol{i_1}, \ldots, \boldsymbol{i_t} \in [1 \,..\, n]$, counters for **variables indices**,
- input relations R_i: clauses incidence, such as
  $R_{+-+--}(i_1, i_2, i_3, i_4, i_5) = \texttt{True} \iff (x_{i_1} \vee \overline{x}_{i_2} \vee x_{i_3} \vee \overline{x}_{i_4} \vee \overline{x}_{i_5}) \in F$,
- solution relation $\boldsymbol{f}$ (just unary!): **a satisfying assignment**, such as
  $f(1) = \texttt{True}$, $f(2) = \texttt{False}, \ldots, f(n) = \texttt{False}$,
- define $f^+(i) = f(i)$ and $f^-(i) = \overline{f(i)}$,
- formula $\Phi(i_1, \ldots, i_t)$:
  $\exists$ assignment f $\;\forall$ indices $i_1, \ldots, i_t \in [1 \,..\, n]$ $\;\forall$ signs $s_1, \ldots s_t \in \{+, -\}$
  $R_{s_1, \ldots, s_t}(i_1, \ldots, i_t)$ // *clause* $(x_{i_1}^{s_1} \vee \ldots) \ldots$
  $\Rightarrow (f^{s_1}(i_1) \vee f^{s_2}(i_2) \vee \ldots \vee f^{s_t}(i_t))$ // *. . . is satisfied*

SNP, still unclear?

Classical form:

$$(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge (\overline{x_1} \vee \overline{x_3}) \wedge (\overline{x_3} \vee \overline{x_2})$$

SNP Input:

$R_{++}(i_1, i_2)$: `False`

$R_{+-}(i_1, i_2)$: if $i_1 = 1$ and $i_2 = 2$ then `True` else `False`

$R_{-+}(i_1, i_2)$: if $i_1 = 1$ and $i_2 = 3$ then `True` else `False`

$R_{--}(i_1, i_2)$: if $(i_1 = 1$ and $i_2 = 3)$ or $(i_1 = 3$ and $i_3 = 2)$ then `True` else `False`

SNP Formula:

$\exists$ assignment $f\colon \{1, 2, 3\} \to \{\texttt{False}, \texttt{True}\}$
$\forall$ indices $i_1, i_2 \in \{1, 2, 3\}$

$$\Big(R_{+,+}(i_1, i_2) \Rightarrow (f(i_1) \vee f(i_2))\Big) \wedge \qquad // \text{ nothing}$$

$$\Big(R_{-,+}(i_1, i_2) \Rightarrow (\overline{f(i_1)} \vee f(i_2))\Big) \wedge \qquad // \ (\neg x_1 \vee x_3)$$

$$\Big(R_{+,-}(i_1, i_2) \Rightarrow (f(i_1) \vee \overline{f(i_2)})\Big) \wedge \qquad // \ (x_1 \vee \neg x_2)$$

$$\Big(R_{-,-}(i_1, i_2) \Rightarrow (\overline{f(i_1)} \vee \overline{f(i_2)})\Big) \qquad // \ (\overline{x_1} \vee \overline{x_3}),\ (\overline{x_3} \vee \overline{x_2})$$

SERF-completeness

Definition (parameterized version of **SNP**)

$(L, q) \in$ **SNP** if L can be defined as

$$(R_1, \ldots, R_u) \in L \iff \exists f_1, \ldots, f_s \; \forall i_1, \ldots, i_t \; \Phi(i_1, \ldots, i_t),$$

where f_j's are relations (arity k_j), $i_j \in [1..d]$, formula Φ applies f_j's and R_j's to i_j's.
$q((R_1, \ldots, R_u))$ is the total bit-size of f_j's, i.e., $\sum_j d^{k_j}$.

- What we need: a class closed under SERF (verify!).
- Every problem is SERF-reducible to ($\boldsymbol{k}$-**SAT**, n):

SERF-completeness

Definition (parameterized version of **SNP**)

$(L, q) \in$ **SNP** if L can be defined as

$$(R_1, \ldots, R_u) \in L \iff \exists f_1, \ldots, f_s \; \forall i_1, \ldots, i_t \; \Phi(i_1, \ldots, i_t),$$

where f_j's are relations (arity k_j), $i_j \in [1..d]$, formula Φ applies f_j's and R_j's to i_j's.
$q((R_1, \ldots, R_u))$ is the total bit-size of f_j's, i.e., $\sum_j d^{k_j}$.

- What we need: a class closed under SERF (verify!).
- Every problem is SERF-reducible to ($\boldsymbol{k}$-**SAT**, n):
  - introduce d^t variables for $f_j(i_1, \ldots, i_t)$, $\rightsquigarrow X_{10;\,1,8,5}$

SERF-completeness

Definition (parameterized version of **SNP**)

$(L, q) \in$ **SNP** if L can be defined as

$$(R_1, \ldots, R_u) \in L \iff \exists f_1, \ldots, f_s \; \forall i_1, \ldots, i_t \; \Phi(i_1, \ldots, i_t),$$

where f_j's are relations (arity k_j), $i_j \in [1..d]$, formula Φ applies f_j's and R_j's to i_j's.
$q((R_1, \ldots, R_u))$ is the total bit-size of f_j's, i.e., $\sum_j d^{k_j}$.

- What we need: a class closed under SERF (verify!).
- Every problem is SERF-reducible to ($\boldsymbol{k}$-**SAT**, n):
  - introduce d^t variables for $f_j(i_1, \ldots, i_t)$, $\rightsquigarrow X_{10;\,1,8,5}$
  - k: how many occurrences of f_j's in Φ,
  - for specific $i_1, i_2, \ldots$ expand R_j's and write Φ as k-CNF of $\leqslant 2^k$ clauses,

SERF-completeness

Definition (parameterized version of **SNP**)

$(L, q) \in$ **SNP** if L can be defined as

$$(R_1, \ldots, R_u) \in L \iff \exists f_1, \ldots, f_s \; \forall i_1, \ldots, i_t \; \Phi(i_1, \ldots, i_t),$$

where f_j's are relations (arity k_j), $i_j \in [1..d]$, formula Φ applies f_j's and R_j's to i_j's.
$q((R_1, \ldots, R_u))$ is the total bit-size of f_j's, i.e., $\sum_j d^{k_j}$.

- What we need: a class closed under SERF (verify!).
- Every problem is SERF-reducible to ($\boldsymbol{k}$-**SAT**, n):
  - introduce d^t variables for $f_j(i_1, \ldots, i_t)$, $\rightsquigarrow X_{10;\,1,8,5}$
  - k: how many occurrences of f_j's in Φ,
  - for specific $i_1, i_2, \ldots$ expand R_j's and write Φ as k-CNF of $\leqslant 2^k$ clauses,
  - replace

    $$\forall i_1, i_2 \ldots \Phi(i_1, i_2, \ldots)$$

    by

    $$\bigwedge_{i_1, i_2 \ldots \in [1..d]} \Phi(i_1, i_2, \ldots).$$

k-SAT vs Unique k-SAT

Lemma

$\forall k\ \forall \varepsilon \in (0, \frac{1}{4})\ s_k \leqslant \sigma_{k'} + O(H(\varepsilon))$, *where* $k' = \max\{k, \frac{1}{\varepsilon} \ln \frac{2}{\varepsilon}\}$.

Corollary

$s_k \leqslant \sigma_k + O(\frac{\ln^2 k}{k})$. *Thus* $s_\infty = \sigma_\infty$.

k-SAT vs Unique k-SAT

Lemma

$\forall k\ \forall \varepsilon \in (0, \frac{1}{4})\ s_k \leqslant \sigma_{k'} + O(H(\varepsilon))$, *where* $k' = \max\{k, \frac{1}{\varepsilon} \ln \frac{2}{\varepsilon}\}$.

Corollary

$s_k \leqslant \sigma_k + O(\frac{\ln^2 k}{k})$. *Thus* $s_\infty = \sigma_\infty$.

Phase 1. Concentration.

Concentrate the satisfying assignments in a ball of radius εn.

How? Add N length-k' random xors (hyperplanes) $\bigoplus_{i \in R} a_i x_i = b$,

where $|R| = k'$ is a random subset; a_i, b_i are random.

Let $k' = \max\{k, \frac{1}{\varepsilon} \ln \frac{2}{\varepsilon}\}$ and $N = O(n)$.

k-SAT vs Unique k-SAT

Lemma

$\forall k\ \forall \varepsilon \in (0, \frac{1}{4})\ s_k \leqslant \sigma_{k'} + O(H(\varepsilon))$, *where* $k' = \max\{k, \frac{1}{\varepsilon} \ln \frac{2}{\varepsilon}\}$.

Corollary

$s_k \leqslant \sigma_k + O(\frac{\ln^2 k}{k})$. *Thus* $s_\infty = \sigma_\infty$.

Phase 1. Concentration.
Concentrate the satisfying assignments in a ball of radius εn.

How? Add N length-k' random xors (hyperplanes) $\bigoplus_{i \in R} a_i x_i = b$,

where $|R| = k'$ is a random subset; a_i, b_i are random.

Let $k' = \max\{k, \frac{1}{\varepsilon} \ln \frac{2}{\varepsilon}\}$ and $N = O(n)$.

Phase 2. Isolation within a ball.
Guess random $S \subseteq [1 \,.\,.\, n]$ with different values.
Guess an assignment for variables in S.

Takeaway

- ETH, SETH.
- $s^{dens.\infty} = s_\infty = \sigma_\infty$.

Takeaway

- ETH, SETH.
- $s^{dens.\infty} = s_\infty = \sigma_\infty$.

Coming next (and next):

- *Proofs of unsatisfiability,*
- *Resolution as a proof system,*
- *Resolution vs DPLL, CDCL,*
- *SAT solvers.*